10주차

날짜	2026년 11월 10일
다중 선택	
텍스트	파머완 6장

6장. 차원 축소

1. 차원 축소(Dimension Reduction) 개요

정의·필요성

- 많은 피처로 구성된 다차원 데이터 세트의 차원을 축소 → 새로운 차원의 데이터 세트 생성
- 차원 증가 시 문제점
  - 데이터 포인트 간 거리가 기하급수적으로 멀어짐 → 희소(sparse) 구조
  - 수백 개 이상 피처 → 예측 신뢰도 하락
  - 피처 간 상관관계 높아짐 → 선형 모델에서 다중 공선성 문제 → 성능 저하
- 차원 축소의 장점
  - 데이터를 더 직관적으로 해석 가능 (3차원 이하로 압축해 시각화)
  - 학습 데이터 크기 감소 → 필요한 처리 능력 감소

피처 선택 vs 피처 추출

- 피처 선택(feature selection)
  - 종속성 강한 불필요 피처는 제거
  - 데이터 특징을 잘 나타내는 주요 피처만 선택
- 피처 추출(feature extraction)
  - 기존 피처를 저차원의 중요 피처로 압축해 추출
  - 새 피처는 기존 피처가 압축된 것 → 기존 피처와 완전히 다른 값
  - 단순 압축이 아니라 피처를 함축적으로 더 잘 설명하는 다른 공간으로 매핑

- 예: 모의고사·내신·수능성적·봉사활동 등 → 학업 성취도, 커뮤니케이션 능력, 문제해결력
- 기존 피처로는 인지하기 어려웠던 **잠재 요소(Latent Factor)** 추출

핵심 의미

- 차원 축소 = 단순 데이터 압축이 아님
- 더 중요한 의미: 데이터를 잘 설명할 수 있는 **잠재적 요소 추출**
- 잠재 요소를 찾는 대표 알고리즘: PCA, SVD, NMF

활용 영역

- 이미지
  - 픽셀이 많은 이미지에서 잠재 특성을 피처로 도출 → 함축적 변환·압축
  - 차원이 훨씬 적어져 과적합(overfitting) 영향 감소 → 오히려 원본보다 예측 성능 향상 가능
  - 원본은 차원이 너무 커서 적은 픽셀 차이도 잘못된 예측으로 이어질 수 있음
- 텍스트
  - 문서 내 단어 구성에 숨겨진 시맨틱(Semantic) 의미·토픽(Topic)을 잠재 요소로 간주해 추출
  - SVD, NMF → 시맨틱 토픽 모델링의 기반 알고리즘

---

2. PCA(Principal Component Analysis)

PCA 개요

기본 개념

- 가장 대표적인 차원 축소 기법
- 여러 변수 간 존재하는 **상관관계**를 이용해 대표하는 **주성분(Principal Component)** 추출 → 차원 축소
- 목표: 기존 데이터의 정보 유실 최소화
- 방법: **가장 높은 분산**을 가지는 데이터 축을 찾아 그 축으로 차원 축소 (분산 = 데이터 특성을 가장 잘 나타내는 것으로 간주)
- 예: 키·몸무게 2개 피처 → 변동성이 가장 큰 방향으로 축 생성 → 새 축에 데이터 투영 → 1개 주성분

축 생성 방식

- 첫 번째 축: 가장 큰 데이터 변동성(Variance) 방향
- 두 번째 축: 첫 번째 축에 직각(직교 벡터)
- 세 번째 축: 두 번째 축과 직각 ... 반복
- 생성된 벡터 축에 원본 데이터 투영 → 벡터 축 개수만큼의 차원으로 축소
- 결과: 원본 피처 수보다 매우 적은 주성분으로 총 변동성 대부분 설명

선형대수 관점

- 입력 데이터의 **공분산 행렬(Covariance Matrix)을 고유값 분해** → 구한 고유벡터에 입력 데이터를 선형 변환
- 고유벡터 = PCA의 주성분 벡터 (분산이 큰 방향)
- 고윳값(eigenvalue) = 고유벡터의 크기 = 입력 데이터의 분산

선형 변환·공분산·고유벡터 용어

- 선형 변환: 특정 벡터에 행렬 A를 곱해 새 벡터로 변환 (한 공간 → 다른 공간으로 투영하는 개념)
- 분산: 1개 변수의 데이터 변동
- 공분산: 2개 변수 간의 변동
  - Cov(X, Y) > 0 → X가 증가할 때 Y도 증가
- 공분산 행렬: 여러 변수의 공분산을 포함하는 정방행렬
  - 대각선 원소 = 각 변수의 분산
  - 대각선 이외 원소 = 모든 변수 쌍의 공분산
- 고유벡터: 행렬 A를 곱해도 **방향은 변하지 않고 크기만 변하는** 벡터
  - Ax = ax (A: 행렬, x: 고유벡터, a: 스칼라값)
  - 정방행렬은 최대 차원 수만큼의 고유벡터 보유 (2x2 → 2개, 3x3 → 3개)
  - 행렬이 작용하는 힘의 방향과 관계 → 행렬 분해에 사용

공분산 행렬의 특성

- 정방행렬(Square Matrix) + 대칭행렬(Symmetric Matrix, $A^T = A$)
- 대칭행렬은 항상 고유벡터를 **직교행렬(orthogonal matrix)**로, 고유값을 정방행렬로 대각화 가능

- 공분산 행렬 C의 분해: **$C = P\Sigma P^T$**
  - P: n×n 직교행렬 (고유벡터 행렬)
  - Σ: n×n 정방행렬 (고유값 대각 행렬)
  - P^T: P의 전치행렬
- 즉 C = 고유벡터 직교행렬 × 고유값 정방행렬 × 고유벡터 직교행렬의 전치행렬
  - e_i: i번째 고유벡터, λ_i: i번째 고유벡터의 크기
  - e_1: 분산이 가장 큰 방향의 고유벡터, e_2: e_1에 수직이면서 그다음으로 분산이 큰 방향
- **핵심**: 공분산 행렬이 고유벡터·고유값으로 분해되고, 이 고유벡터로 입력 데이터를 선형 변환하는 방식이 PCA

PCA 수행 4단계

1. 입력 데이터 세트의 공분산 행렬 생성
2. 공분산 행렬의 고유벡터와 고유값 계산
3. 고유값이 가장 큰 순으로 K개(PCA 변환 차수만큼) 고유벡터 추출
4. 추출된 고유벡터로 입력 데이터 변환

붓꽃(Iris) 데이터 PCA 예제

목표: 4개 속성(sepal length/width, petal length/width) → 2개 PCA 차원으로 압축

원본 데이터 시각화

- sepal length(X축), sepal width(Y축) 산점도
- Setosa: sepal width > 3.0, sepal length ≤ 6.0 영역에 일정하게 분포
- Versicolor와 Virginica: 두 속성만으로는 분류가 어려움

PCA 적용 절차

- **PCA 적용 전 반드시 스케일링**
  - PCA는 여러 속성값을 연산하므로 속성 스케일의 영향을 받음
  - StandardScaler로 평균 0, 분산 1인 표준 정규분포로 변환
- `PCA(n_components=2)` → `fit()` → `transform()` → (150, 2) 반환

주요 함수·속성

- `StandardScaler().fit_transform()`: 표준화

- `PCA(n_components=)` : 변환할 차원 수 지정
- `pca.fit()` , `pca.transform()` : 변환 수행
- `pca.explained_variance_ratio_` : 각 PCA 컴포넌트가 차지하는 변동성 비율

결과 해석

- PCA 변환 후에도 pca_component_1 축 기준으로 Setosa 명확히 구분
- Versicolor/Virginica는 일부 겹치지만 비교적 잘 구분
  - 이유: 첫 번째 축이 원본 데이터 변동성을 잘 반영
- explained_variance_ratio_: [0.7296, 0.2285]
  - 첫 번째 약 72.9%, 두 번째 약 22.8% → 2개 요소로 원본 변동성 약 95% 설명

분류 성능 비교 (RandomForest, cross_val_score, cv=3)

구분	평균 정확도
원본 데이터(4개 속성)	0.96
PCA 변환(2개 속성)	0.88

- PCA 차원 개수에 따라 예측 성능은 떨어질 수밖에 없음
- 약 8% 하락했지만, 속성이 50% 감소한 점을 고려하면 원본 특성을 상당 부분 유지

신용카드 고객 데이터 PCA 예제

데이터 세트

- UCI ML Repository의 Credit Card Clients Data Set
- 30,000개 레코드, 24개 속성 (피처 23개)
- 타깃: `default payment next month` (연체 1 / 정상납부 0) → 컬럼명 `default`로 변경
- PAY_0 → PAY_1로 컬럼명 변경

주요 함수

- `pd.read_excel('파일명', header=1, sheet_name='Data')` : 엑셀 로드 (header=1로 의미 없는 첫 행 제거)
- `.iloc[0:, 1:]` : 기존 id 컬럼 제거
- `df.rename(columns={}, inplace=True)` : 컬럼명 변경
- `df.corr()` + `sns.heatmap()` : 속성 간 상관도 시각화

상관도 분석

- BILL_AMT1 ~ BILL_AMT6: 상관도 대부분 0.9 이상 (매우 높음)
- PAY_1 ~ PAY_6: 이보다 낮지만 상관도 높음
- 상관도가 높은 속성들은 소수의 PCA만으로도 변동성을 자연스럽게 수용 가능

BILL_AMT1~6 → PCA 2개 컴포넌트

- explained_variance_ratio_: [0.9056, 0.0510]
- 2개 컴포넌트로 95% 이상 설명, 첫 번째 축 하나로 약 90% 수용

분류 성능 비교 (RandomForest n_estimators=300, cv=3)

구분	평균 정확도
원본 데이터(23개 속성)	0.8170
PCA 변환(6개 컴포넌트)	0.7973

- 속성의 약 1/4(6개)만으로 약 1~2% 성능 저하에 그침 → PCA의 뛰어난 압축 능력
- 원본 전체에 StandardScaler 적용 후 PCA(n_components=6) 수행

PCA 활용 분야

- 차원 축소로 데이터를 쉽게 인지
- 더 활발한 영역: 컴퓨터 비전(Computer Vision)
- 얼굴 인식: **Eigen-face**라 불리는 PCA 변환을 사용하는 경우가 많음

---

3. LDA(Linear Discriminant Analysis)

LDA 개요

기본 개념

- 선형 판별 분석법, PCA와 매우 유사
- 입력 데이터 세트를 저차원 공간에 투영해 차원 축소
- **PCA와의 차이**
  - PCA: 입력 데이터 **변동성**이 가장 큰 축을 찾음
  - LDA: 입력 데이터의 **결정 값 클래스를 최대한 분리**할 수 있는 축을 찾음
  - LDA는 **지도학습**의 분류에서 사용하기 쉽도록 개별 클래스를 분별할 수 있는 기준을 최대한 유지

원리

- 클래스 간 분산(between-class scatter)과 클래스 내부 분산(within-class scatter)의 **비율을 최대화**
- 클래스 간 분산 ↑(최대한 크게), 클래스 내부 분산 ↓(최대한 작게)

PCA와 구하는 방식의 차이

- PCA: 공분산 행렬 사용
- LDA: **클래스 간 분산 행렬 + 클래스 내부 분산 행렬** 생성 후 고유벡터 도출

LDA 수행 4단계

1. 클래스 내부 / 클래스 간 분산 행렬 구하기 (입력 데이터의 결정 값 클래스별로 개별 피처의 평균 벡터(mean vector) 기반)
2. 클래스 내부 분산 행렬 S_W, 클래스 간 분산 행렬 S_B → S_W^T S_B를 고유벡터로 분해
3. 고유값이 가장 큰 순으로 K개(LDA 변환 차수만큼) 추출
4. 추출된 고유벡터로 입력 데이터 변환

붓꽃 데이터 세트에 LDA 적용하기

주요 함수

- `LinearDiscriminantAnalysis(n_components=2)` : sklearn의 LDA 클래스
- `lda.fit(iris_scaled, iris.target)` : **결정값(타깃)을 함께 입력**
- `lda.transform(iris_scaled)` : 변환

핵심 포인트

- LDA는 PCA와 달리 **비지도학습이 아닌 지도학습** → 클래스 결정값이 변환 시 필요
- 붓꽃 데이터는 로드 후 표준 정규분포로 스케일링해서 사용
- 결과 시각화: PCA 변환 데이터와 **좌우 대칭 형태**로 많이 닮음

---

4. SVD(Singular Value Decomposition)

SVD 개요

기본 개념

- PCA와 유사한 행렬 분해 기법
- **PCA와의 차이**

    - PCA: 정방행렬(행·열 크기가 같은 행렬)만 고유벡터로 분해 가능
    - SVD: 정방행렬뿐 아니라 **행과 열 크기가 다른 행렬에도 적용 가능**
- 특이값 분해라 부름

SVD 분해식: $A = U\Sigma V^T$ (A: m×n)

- U: m×m, Σ: m×n, V^T: n×n
- U, V에 속한 벡터: **특이벡터(singular vector)**, 모든 특이벡터는 서로 직교
- Σ: 대각행렬, 대각에 위치한 값만 0이 아니고 나머지는 모두 0
    - Σ의 0이 아닌 대각 값 = 행렬 A의 **특이값**

컴팩트한 형태의 SVD

- Σ의 비대각 부분과, 대각원소 중 특이값이 0인 부분을 모두 제거
- 제거된 Σ에 대응하는 U, V 원소도 함께 제거
- 차원: U(m×p), Σ(p×p), V^T(p×n)

Truncated SVD

- Σ의 대각원소 중 **상위 일부 개수만 추출**, 여기에 대응하는 U, V 원소도 제거 → 더욱 차원을 줄인 형태
- 인위적으로 더 작은 차원으로 분해 → **원본 행렬을 정확히 복원할 수 없음** (근사)
- 그래도 상당한 수준으로 원본 행렬 근사 가능
- 원래 차원 차수에 가깝게 자를수록 원본에 더 가깝게 복원

넘파이 SVD 실습

- `numpy.linalg.svd(a)` → U, Sigma, Vt 반환
    - Sigma는 0이 아닌 값만 **1차원 배열**로 반환 → 복원 시 `np.diag(Sigma)`로 대각행렬 변환 후 내적
    - 복원: `np.dot(np.dot(U, Sigma_mat), Vt)` → 원본과 동일하게 복원됨
- 4×4 랜덤 행렬: U(4×4), Sigma(4,), Vt(4×4)
- 랜덤 행렬을 쓴 이유: 행렬의 개별 로우끼리의 의존성을 없애기 위해

로우 간 의존성이 있는 경우

- 3번째 로우 = 1번째 + 2번째 로우, 4번째 로우 = 1번째 로우로 의존성 부여

- Sigma 값 중 **2개가 0으로 변함** → 선형 독립인 로우 벡터 개수 = 2 (행렬의 랭크(Rank)가 2)
- Sigma 0 대응 부분 제외 복원
  - U는 앞 2열, Sigma는 앞 2개, Vt는 앞 2행만 추출 → (4,2), (2,2), (2,4)
  - 원본 행렬로 정확히 복원됨

Truncated SVD 실습 (사이파이)

- 넘파이는 미지원, **사이파이에서만 지원**
- 일반 SVD: `scipy.linalg.svd`
- Truncated SVD: `scipy.sparse.linalg.svds` (**희소 행렬 전용**)
- 6×6 행렬 예시 (`svds(matrix, k=4)`)
  - 일반 SVD: U(6,6), Sigma(6,), Vt(6,6)
  - Truncated SVD: U(6,4), Sigma(4,), Vt(4,6)
- Truncated SVD로 복원 시 **완벽하지 않고 근사적으로 복원**

사이킷런 TruncatedSVD 클래스를 이용한 변환

- 사이파이 svds처럼 Truncated SVD 연산을 수행하지만 **U, Sigma, Vt를 반환하지 않음**
- PCA 클래스와 유사하게 `fit()`, `transform()`으로 사용
- 원본 데이터를 Truncated SVD 방식으로 분해된 **U×Sigma 행렬에 선형 변환**해 생성
- `TruncatedSVD(n_components=2)`: 주요 컴포넌트 수(K) 지정
- 붓꽃 데이터 변환 결과: PCA와 유사하게 품종별 클러스터링이 가능할 정도의 고유성

PCA와 TruncatedSVD 비교

- 두 클래스 모두 내부적으로 SVD를 이용해 행렬 분해
- 스케일링(StandardScaler) 후 변환하면 **거의 동일한 결과**
  - 두 변환 행렬의 평균 차이가 0에 가까운 값 (~e-15, ~e-17)
- 즉 데이터가 스케일링으로 중심이 동일해지면 사이킷런의 SVD와 PCA는 **동일한 변환** 수행 → PCA가 SVD 알고리즘으로 구현됨
- 차이: PCA는 **밀집 행렬(Dense Matrix)**만 변환 가능, SVD는 **희소 행렬(Sparse Matrix)**도 변환 가능

활용

- 컴퓨터 비전 영역에서 이미지 압축을 통한 패턴 인식, 신호 처리 분야
- 텍스트 토픽 모델링 기법인 **LSA(Latent Semantic Analysis)**의 기반 알고리즘

---

5. NMF(Non-Negative Matrix Factorization)

NMF 개요

기본 개념

- Truncated SVD처럼 **낮은 랭크를 통한 행렬 근사(Low-Rank Approximation)** 방식의 변형
- 원본 행렬 내 **모든 원소 값이 양수(0 이상)**임이 보장될 때, 두 개의 기반 양수 행렬로 분해하는 기법
- 예: 4×6 원본 행렬 V ≈ 4×2 행렬 W × 2×6 행렬 H

행렬 분해(Matrix Factorization)

- SVD 같은 행렬 분해 기법을 통칭
- W: 길고 가는 행렬 (행 크기는 원본과 같고 열 크기는 원본보다 작음)
- H: 작고 넓은 행렬 (행 크기는 원본보다 작고 열 크기는 원본과 같음)
- 분해된 행렬은 **잠재 요소(Latent Factor)**를 특성으로 가짐
  - **W**: 원본 행에 대해 잠재 요소의 값이 얼마나 되는지에 대응
  - **H**: 잠재 요소가 원본 열(속성)로 어떻게 구성됐는지 나타냄

사이킷런 NMF

- `NMF(n_components=2)` → `fit()` → `transform()`
- 붓꽃 데이터 2개 컴포넌트로 변환 시 품종별 구분 가능

활용 분야

- 이미지 압축을 통한 패턴 인식
- 텍스트 토픽 모델링, 문서 유사도 및 클러스터링
- **추천(Recommendations)**: 사용자-평가 순위(user-Rating) 데이터 세트를 행렬 분해 → 사용자가 평가하지 않은 상품의 잠재 요소를 추출 → 평가 순위 예측 → 높은 순위 상품 추천
  - 이를 **잠재 요소(Latent Factoring) 기반의 추천 방식**이라 함